

```
# =====
# Chess Game Outcome Predictor - Deep Neural Network
# MTH 3320 / CSC 2590 - Final Project
# =====
# Dataset: Chess Game Dataset (Lichess) from Kaggle
# https://www.kaggle.com/datasets/datasnaek/chess
#
# SETUP:
# !pip install kaggle
# Upload kaggle.json API token, then:
# !kaggle datasets download -d datasnaek/chess
# !unzip chess.zip
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, classification_report

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42)
tf.random.set_seed(42)
```

```
# =====
# Download & Unzip the Dataset
# =====
# Run commands once to download and unzip the dataset
!pip install kaggle

import os
if not os.path.exists('games.csv'):
    print("Downloading and unzipping chess dataset...")
    !kaggle datasets download -d datasnaek/chess -p .
    !unzip -o chess.zip
    print("Dataset downloaded and unzipped.")
```

```
else:
    print("games.csv already exists.")
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: jupyterlab in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: kagglesdk<1.0, >=0.1.20 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: markdown-it-py>=1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mdit-py-plugins in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mdurl<=0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: jupyter-core!=5.0.*, >=4.12 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local/lib/python3.12/dist-packages
games.csv already exists.
```

1. LOAD & EXPLORE DATA

```
df = pd.read_csv("games.csv")

print("Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nFirst 3 rows:")
print(df.head(3))
print("\nMissing values:")
print(df.isnull().sum())
```

```

# Class distribution
print("\nOutcome distribution:")
print(df["winner"].value_counts())

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Winner distribution
df["winner"].value_counts().plot(kind="bar", ax=axes[0], color=["#4CAF50", "#F44336", "#FFEB3B"])
axes[0].set_title("Game Outcome Distribution")
axes[0].set_xlabel("Winner")
axes[0].set_ylabel("Count")
axes[0].tick_params(axis="x", rotation=0)

# ELO distributions
axes[1].hist(df["white_rating"], bins=40, alpha=0.6, label="White ELO", color="#4CAF50")
axes[1].hist(df["black_rating"], bins=40, alpha=0.6, label="Black ELO", color="#F44336")
axes[1].set_title("Player ELO Distribution")
axes[1].set_xlabel("ELO Rating")
axes[1].set_ylabel("Count")
axes[1].legend()

# ELO difference vs outcome
df["elo_diff"] = df["white_rating"] - df["black_rating"]
for outcome, color in zip(["white", "black", "draw"], ["#42A5F5", "#EF5350", "#FFEB3B"]):
    subset = df[df["winner"] == outcome]["elo_diff"]
    axes[2].hist(subset, bins=40, alpha=0.6, label=outcome, color=color)
axes[2].set_title("ELO Difference by Outcome")
axes[2].set_xlabel("White ELO - Black ELO")
axes[2].set_ylabel("Count")
axes[2].legend()

plt.tight_layout()
plt.savefig("eda_plots.png", dpi=150)
plt.show()
print("EDA plots saved.")

```

Shape: (20058, 16)

Columns: ['id', 'rated', 'created_at', 'last_move_at', 'turns', 'victory_status',

First 3 rows:

	id	rated	created_at	last_move_at	turns	victory_status	winner	\
0	TZJHLljE	False	1.504210e+12	1.504210e+12	13	outoftime	white	
1	l1NXvwaE	True	1.504130e+12	1.504130e+12	16	resign	black	
2	mIICvQhH	True	1.504130e+12	1.504130e+12	61	mate	white	

	increment_code	white_id	white_rating	black_id	black_rating	\
0	1542	houzaris	1500	2000	1100	

2. PREPROCESSING

```
# Drop rows with missing winner
df = df.dropna(subset=["winner"])

# Features we'll use
features = [
    "white_rating",
    "black_rating",
    "elo_diff",
    "turns",
    "opening_name",
    "increment_code",
    "rated",
]

df = df[features + ["winner"]].copy()

# --- Encode 'rated' (True/False → 1/0) ---
df["rated"] = df["rated"].astype(int)

# --- Extract time control base from increment_code (e.g. "10+0" → 10)
def parse_time(code):
    try:
        return int(str(code).split("+")[0])
    except:
        return 0

df["time_base"] = df["increment_code"].apply(parse_time)

# --- Simplify opening names to first segment ---
df["opening_family"] = df["opening_name"].apply(
    lambda x: str(x).split(":")[0].split("|")[0].strip()
)

# Keep only the top 20 openings, group the rest as "Other"
top_openings = df["opening_family"].value_counts().nlargest(20).index
df["opening_family"] = df["opening_family"].apply(
```

```

        lambda x: x if x in top_openings else "Other"
    )

# --- Label encode opening family ---
le_opening = LabelEncoder()
df["opening_encoded"] = le_opening.fit_transform(df["opening_family"])

# --- Encode target ---
le_target = LabelEncoder()
df["outcome"] = le_target.fit_transform(df["winner"]) # black=0, draw=1, white=2
print("Class mapping:", dict(zip(le_target.classes_, le_target.transform(le_target.classes_)))

# --- Final feature matrix ---
X = df[["white_rating", "black_rating", "elo_diff", "turns", "rated",
y = df["outcome"].values

# --- Train/val/test split: 70/15/15 ---
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.15, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

print(f"\nTrain: {X_train.shape[0]} | Val: {X_val.shape[0]} | Test: {X_test.shape[0]}")

# --- Normalize ---
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

# One-hot encode labels
y_train_oh = keras.utils.to_categorical(y_train, num_classes=3)
y_val_oh = keras.utils.to_categorical(y_val, num_classes=3)
y_test_oh = keras.utils.to_categorical(y_test, num_classes=3)

```

```

Class mapping: {'black': np.int64(0), 'draw': np.int64(1), 'white': np.int64(2)}
Train: 14040 | Val: 3009 | Test: 3009

```

3. MODEL 1 - BASELINE DNN

```

def build_baseline():
    model = keras.Sequential([
        layers.Input(shape=(7,)),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(8, activation="relu"),
        layers.Dense(4, activation="relu"),
        layers.Dense(2, activation="softmax")
    ])
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])

```

```

        layers.Dense(3, activation="softmax"),
    ], name="Baseline_DNN")
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return model

baseline = build_baseline()
baseline.summary()

history_baseline = baseline.fit(
    X_train, y_train_oh,
    validation_data=(X_val, y_val_oh),
    epochs=50,
    batch_size=64,
    verbose=1,
)

baseline_loss, baseline_acc = baseline.evaluate(X_test, y_test_oh, verbose=0)
print(f"\nBaseline Test Accuracy: {baseline_acc:.4f}")

```

Model: "Baseline_DNN"

Layer (type)	Output Shape	Param #
dense_7 (Dense)	(None, 32)	1024
dense_8 (Dense)	(None, 16)	528
dense_9 (Dense)	(None, 3)	48

Total params: 835 (3.26 KB)
Trainable params: 835 (3.26 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/50

220/220 ————— **15s** 22ms/step - accuracy: 0.5897 - loss: 0.8236 - val_loss: 0.7622

Epoch 2/50

220/220 ————— **4s** 18ms/step - accuracy: 0.6244 - loss: 0.7622 - val_loss: 0.7574

Epoch 3/50

220/220 ————— **4s** 16ms/step - accuracy: 0.6264 - loss: 0.7574 - val_loss: 0.7528

Epoch 4/50

220/220 ————— **3s** 13ms/step - accuracy: 0.6256 - loss: 0.7547 - val_loss: 0.7514

Epoch 5/50

220/220 ————— **4s** 19ms/step - accuracy: 0.6268 - loss: 0.7528 - val_loss: 0.7514

Epoch 6/50

220/220 ————— **2s** 10ms/step - accuracy: 0.6271 - loss: 0.7514 - val_loss: 0.7503

Epoch 7/50

220/220 ————— **2s** 10ms/step - accuracy: 0.6276 - loss: 0.7503 - val_loss: 0.7493

Epoch 8/50

220/220 ————— **3s** 12ms/step - accuracy: 0.6280 - loss: 0.7493 - val_loss: 0.7486

Epoch 9/50

220/220 ————— **2s** 7ms/step - accuracy: 0.6281 - loss: 0.7486 - val_loss: 0.7479

Epoch 10/50

220/220 ————— **2s** 7ms/step - accuracy: 0.6277 - loss: 0.7479 - val_loss: 0.7479

Epoch 11/50

```

220/220 ————— 3s 9ms/step - accuracy: 0.6278 - loss: 0.7474 - val_
Epoch 12/50
220/220 ————— 1s 6ms/step - accuracy: 0.6293 - loss: 0.7469 - val_
Epoch 13/50
220/220 ————— 2s 5ms/step - accuracy: 0.6296 - loss: 0.7465 - val_
Epoch 14/50
220/220 ————— 1s 2ms/step - accuracy: 0.6292 - loss: 0.7461 - val_
Epoch 15/50
220/220 ————— 1s 2ms/step - accuracy: 0.6293 - loss: 0.7457 - val_
Epoch 16/50
220/220 ————— 1s 2ms/step - accuracy: 0.6296 - loss: 0.7454 - val_
Epoch 17/50
220/220 ————— 1s 2ms/step - accuracy: 0.6290 - loss: 0.7451 - val_
Epoch 18/50
220/220 ————— 1s 2ms/step - accuracy: 0.6293 - loss: 0.7447 - val_
Epoch 19/50
220/220 ————— 1s 2ms/step - accuracy: 0.6293 - loss: 0.7445 - val_
Epoch 20/50
220/220 ————— 1s 2ms/step - accuracy: 0.6283 - loss: 0.7442 - val_
Epoch 21/50
220/220 ————— 1s 2ms/step - accuracy: 0.6288 - loss: 0.7440 - val_
Epoch 22/50
220/220 ————— 1s 2ms/step - accuracy: 0.6288 - loss: 0.7437 - val_
Epoch 23/50
220/220 ————— 1s 2ms/step - accuracy: 0.6285 - loss: 0.7435 - val_
Epoch 24/50
220/220 ————— 1s 2ms/step - accuracy: 0.6286 - loss: 0.7432 - val_
Epoch 25/50
220/220 ————— 1s 2ms/step - accuracy: 0.6292 - loss: 0.7430 - val_
Epoch 26/50
220/220 ————— 1s 4ms/step - accuracy: 0.6291 - loss: 0.7427 - val_
Epoch 27/50
220/220 ————— 1s 4ms/step - accuracy: 0.6281 - loss: 0.7425 - val_
Epoch 28/50
220/220 ————— 1s 4ms/step - accuracy: 0.6288 - loss: 0.7423 - val_
Epoch 29/50
220/220 ————— 1s 2ms/step - accuracy: 0.6291 - loss: 0.7421 - val_
Epoch 30/50
220/220 ————— 1s 2ms/step - accuracy: 0.6291 - loss: 0.7419 - val_
Epoch 31/50
220/220 ————— 1s 2ms/step - accuracy: 0.6293 - loss: 0.7417 - val_
Epoch 32/50
220/220 ————— 1s 2ms/step - accuracy: 0.6301 - loss: 0.7415 - val_
Epoch 33/50
220/220 ————— 1s 2ms/step - accuracy: 0.6303 - loss: 0.7413 - val_
Epoch 34/50
220/220 ————— 1s 2ms/step - accuracy: 0.6300 - loss: 0.7411 - val_
Epoch 35/50
220/220 ————— 1s 2ms/step - accuracy: 0.6299 - loss: 0.7410 - val_
Epoch 36/50
220/220 ————— 1s 2ms/step - accuracy: 0.6299 - loss: 0.7408 - val_
Epoch 37/50
220/220 ————— 1s 2ms/step - accuracy: 0.6303 - loss: 0.7407 - val_
Epoch 38/50
220/220 ————— 1s 2ms/step - accuracy: 0.6298 - loss: 0.7405 - val_
Epoch 39/50
220/220 ————— 1s 2ms/step - accuracy: 0.6295 - loss: 0.7403 - val_
Epoch 40/50

```



```

220/220 ————— 1s 2ms/step - accuracy: 0.6300 - loss: 0.7401 - val_
Epoch 41/50
220/220 ————— 1s 2ms/step - accuracy: 0.6298 - loss: 0.7399 - val_
Epoch 42/50
220/220 ————— 1s 2ms/step - accuracy: 0.6304 - loss: 0.7398 - val_
Epoch 43/50
220/220 ————— 1s 2ms/step - accuracy: 0.6296 - loss: 0.7397 - val_
Epoch 44/50
220/220 ————— 1s 2ms/step - accuracy: 0.6300 - loss: 0.7395 - val_
Epoch 45/50
220/220 ————— 1s 2ms/step - accuracy: 0.6298 - loss: 0.7394 - val_
Epoch 46/50
220/220 ————— 1s 2ms/step - accuracy: 0.6308 - loss: 0.7392 - val_
Epoch 47/50
220/220 ————— 1s 4ms/step - accuracy: 0.6309 - loss: 0.7391 - val_
Epoch 48/50
220/220 ————— 1s 4ms/step - accuracy: 0.6308 - loss: 0.7389 - val_
Epoch 49/50
220/220 ————— 1s 4ms/step - accuracy: 0.6307 - loss: 0.7388 - val_
Epoch 50/50
220/220 ————— 1s 3ms/step - accuracy: 0.6305 - loss: 0.7387 - val_

```

Baseline Test Accuracy: 0.6075

4. MODEL 2 – IMPROVED DNN

```

def build_improved():
    model = keras.Sequential([
        layers.Input(shape=(7,)),
        layers.Dense(128, activation="relu"),
        layers.BatchNormalization(),
        layers.Dropout(0.3),
        layers.Dense(64, activation="relu"),
        layers.BatchNormalization(),
        layers.Dropout(0.3),
        layers.Dense(32, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(3, activation="softmax"),
    ])

```

```

    ], name="Improved_DNN")
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

improved = build_improved()
improved.summary()

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=8, restore_best_weights=True
)

history_improved = improved.fit(
    X_train, y_train_oh,
    validation_data=(X_val, y_val_oh),
    epochs=100,
    batch_size=64,

    callbacks=[early_stop],
    verbose=1,
)

improved_loss, improved_acc = improved.evaluate(X_test, y_test_oh, verbose=0)
print(f"\nImproved Model Test Accuracy: {improved_acc:.4f}")

```

Model: "Improved_DNN"

Layer (type)	Output Shape	Param #
dense_10 (Dense)	(None, 128)	13152
batch_normalization_2 (Batch Normalization)	(None, 128)	128
dropout_3 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 64)	8320
batch_normalization_3 (Batch Normalization)	(None, 64)	128
dropout_4 (Dropout)	(None, 64)	0
dense_12 (Dense)	(None, 32)	2112
dropout_5 (Dropout)	(None, 32)	0
dense_13 (Dense)	(None, 3)	99

Total params: 12,227 (47.76 KB)
Trainable params: 11,843 (46.26 KB)
Non-trainable params: 384 (1.50 KB)

```
Epoch 1/100
220/220 ————— 3s 4ms/step - accuracy: 0.5519 - loss: 0.9482 - val_
Epoch 2/100
220/220 ————— 1s 3ms/step - accuracy: 0.5869 - loss: 0.8152 - val_
Epoch 3/100
220/220 ————— 1s 3ms/step - accuracy: 0.5978 - loss: 0.7966 - val_
Epoch 4/100
220/220 ————— 1s 5ms/step - accuracy: 0.6038 - loss: 0.7812 - val_
Epoch 5/100
220/220 ————— 1s 5ms/step - accuracy: 0.6132 - loss: 0.7741 - val_
Epoch 6/100
220/220 ————— 1s 5ms/step - accuracy: 0.6145 - loss: 0.7735 - val_
Epoch 7/100
220/220 ————— 1s 4ms/step - accuracy: 0.6214 - loss: 0.7662 - val_
Epoch 8/100
220/220 ————— 1s 3ms/step - accuracy: 0.6199 - loss: 0.7670 - val_
Epoch 9/100
220/220 ————— 1s 3ms/step - accuracy: 0.6205 - loss: 0.7616 - val_
Epoch 10/100
220/220 ————— 1s 3ms/step - accuracy: 0.6226 - loss: 0.7632 - val_
Epoch 11/100
220/220 ————— 1s 3ms/step - accuracy: 0.6233 - loss: 0.7608 - val_
Epoch 12/100
220/220 ————— 1s 3ms/step - accuracy: 0.6199 - loss: 0.7615 - val_
Epoch 13/100
220/220 ————— 1s 3ms/step - accuracy: 0.6247 - loss: 0.7611 - val_
Epoch 14/100
220/220 ————— 1s 3ms/step - accuracy: 0.6215 - loss: 0.7597 - val_
Epoch 15/100
220/220 ————— 1s 3ms/step - accuracy: 0.6222 - loss: 0.7604 - val_
Epoch 16/100
220/220 ————— 1s 3ms/step - accuracy: 0.6230 - loss: 0.7569 - val_
Epoch 17/100
220/220 ————— 1s 3ms/step - accuracy: 0.6245 - loss: 0.7579 - val_
Epoch 18/100
220/220 ————— 2s 8ms/step - accuracy: 0.6236 - loss: 0.7560 - val_
Epoch 19/100
220/220 ————— 2s 6ms/step - accuracy: 0.6236 - loss: 0.7579 - val_
Epoch 20/100
220/220 ————— 1s 4ms/step - accuracy: 0.6278 - loss: 0.7572 - val_
Epoch 21/100
220/220 ————— 1s 3ms/step - accuracy: 0.6223 - loss: 0.7552 - val_
Epoch 22/100
220/220 ————— 1s 3ms/step - accuracy: 0.6231 - loss: 0.7556 - val_
Epoch 23/100
220/220 ————— 1s 3ms/step - accuracy: 0.6236 - loss: 0.7540 - val_
Epoch 24/100
220/220 ————— 1s 3ms/step - accuracy: 0.6236 - loss: 0.7558 - val_
Epoch 25/100
220/220 ————— 1s 5ms/step - accuracy: 0.6267 - loss: 0.7525 - val_
Epoch 26/100
220/220 ————— 1s 3ms/step - accuracy: 0.6268 - loss: 0.7550 - val_
Epoch 27/100
220/220 ————— 1s 3ms/step - accuracy: 0.6236 - loss: 0.7551 - val_
```

Improved Model Test Accuracy: 0.6132

5. TRAINING CURVES

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Baseline
axes[0, 0].plot(history_baseline.history["accuracy"], label="Train")
axes[0, 0].plot(history_baseline.history["val_accuracy"], label="Val")
axes[0, 0].set_title("Baseline - Accuracy")
axes[0, 0].set_xlabel("Epoch")
axes[0, 0].legend()

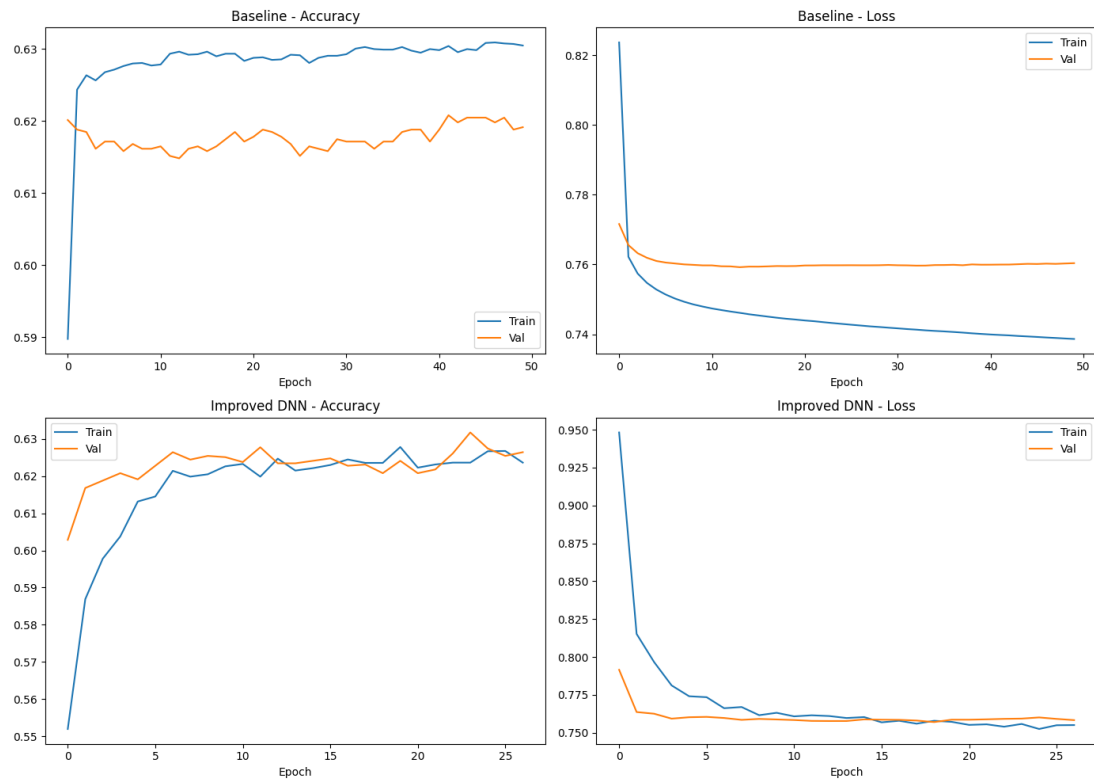
axes[0, 1].plot(history_baseline.history["loss"], label="Train")
axes[0, 1].plot(history_baseline.history["val_loss"], label="Val")
axes[0, 1].set_title("Baseline - Loss")
axes[0, 1].set_xlabel("Epoch")
axes[0, 1].legend()

# Improved
axes[1, 0].plot(history_improved.history["accuracy"], label="Train")
axes[1, 0].plot(history_improved.history["val_accuracy"], label="Val")
axes[1, 0].set_title("Improved DNN - Accuracy")
axes[1, 0].set_xlabel("Epoch")
axes[1, 0].legend()

axes[1, 1].plot(history_improved.history["loss"], label="Train")
axes[1, 1].plot(history_improved.history["val_loss"], label="Val")
axes[1, 1].set_title("Improved DNN - Loss")
axes[1, 1].set_xlabel("Epoch")
axes[1, 1].legend()

plt.tight_layout()
plt.savefig("training_curves.png", dpi=150)
```

```
plt.show()  
print("Training curves saved.")
```



Training curves saved.

6. EVALUATION – CONFUSION MATRIX & CLASSIFICATION REPORT

```

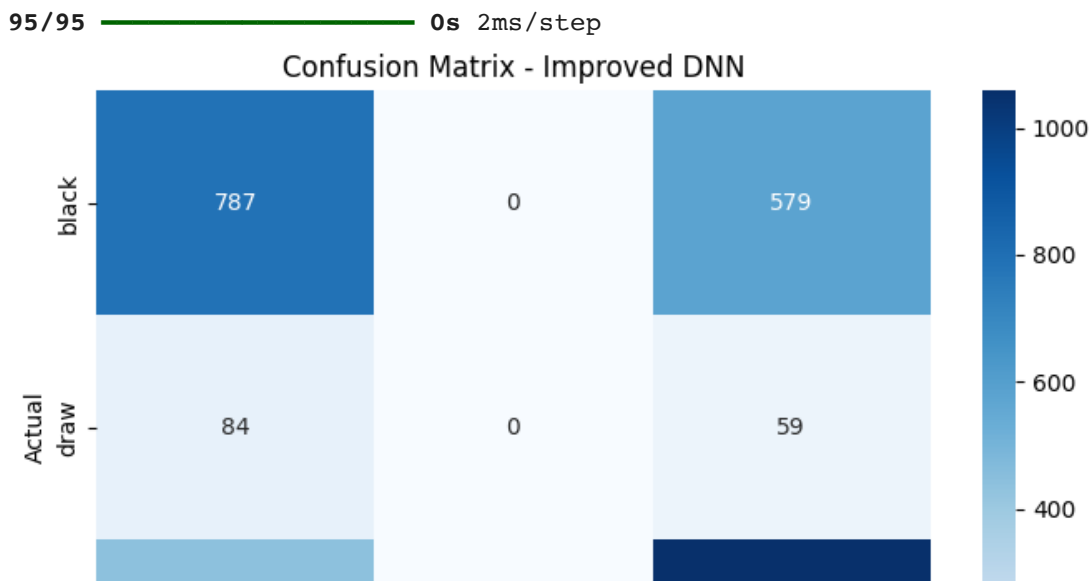
y_pred = np.argmax(improved.predict(X_test), axis=1)
class_names = le_target.classes_ # ['black', 'draw', 'white']

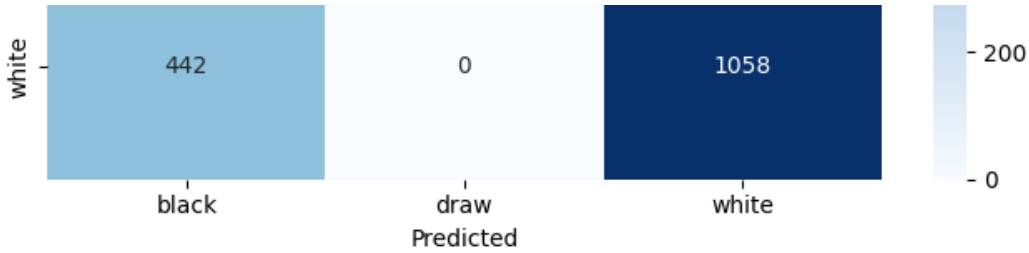
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(7, 5))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=class_names, yticklabels=class_names
)
plt.title("Confusion Matrix – Improved DNN")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()
print("Confusion matrix saved.")

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names))

# Model comparison summary
print("\n--- Model Comparison ---")
print(f"Baseline Accuracy: {baseline_acc:.4f}")
print(f"Improved Accuracy: {improved_acc:.4f}")
print(f"Improvement: +{(improved_acc - baseline_acc):.4f}")

```





Confusion matrix saved.

Classification Report:

	precision	recall	f1-score	support
black	0.60	0.58	0.59	1366
draw	0.00	0.00	0.00	143
white	0.62	0.71	0.66	1500
accuracy			0.61	3009
macro avg	0.41	0.43	0.42	3009
weighted avg	0.58	0.61	0.60	3009

--- Model Comparison ---

Baseline Accuracy: 0.6075

Improved Accuracy: 0.6132

Improvement: +0.0056

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

7. INTERACTIVE PREDICTOR

```

def predict_outcome(white_elo, black_elo, opening_name, turns=40, rated
    """
    Predict chess game outcome probabilities.

    Parameters
    -----
    white_elo    : int    - White player ELO rating

```

```

black_elo      : int    - Black player ELO rating
opening_name   : str    - Opening family name (e.g. "Sicilian Defense"
turns          : int    - Expected number of turns (default 40)
rated          : bool   - Whether the game is rated (default True)
time_base      : int    - Base time in minutes (default 10)

```

Returns

dict with win probabilities for white, black, and draw
 ""

```
elo_diff = white_elo - black_elo
```

```
# Encode opening
```

```
opening_clean = opening_name.strip()
```

```
if opening_clean not in le_opening.classes_:
```

```
    opening_clean = "Other"
```

```
opening_enc = le_opening.transform([opening_clean])[0]
```

```

features = np.array([[
    white_elo, black_elo, elo_diff,
    turns, int(rated), time_base, opening_enc
]])

```

```
features_scaled = scaler.transform(features)
```

```
probs = improved.predict(features_scaled, verbose=0)[0]
```

```

result = {
    "Black wins": round(float(probs[0]) * 100, 1),
    "Draw":      round(float(probs[1]) * 100, 1),
    "White wins": round(float(probs[2]) * 100, 1),
}

```

```
return result
```

```
# --- Example predictions ---
```

```
print("\n--- Interactive Predictor Examples ---\n")
```

```

examples = [
    (1800, 1200, "Sicilian Defense"),
    (1500, 1500, "King's Pawn Game"),
    (2200, 2100, "Queen's Gambit"),
    (1000, 1800, "French Defense"),
]

```

```
for w, b, op in examples:
```

```
    result = predict_outcome(w, b, op)
```

```
    print(f"White ELO: {w} | Black ELO: {b} | Opening: {op}")
```

```
    print(f"Black win prob: {result['Black wins']}%")
```

```
    print(f"Draw prob: {result['Draw']}%")
```

```
    print(f"White win prob: {result['White wins']}%")
```



```

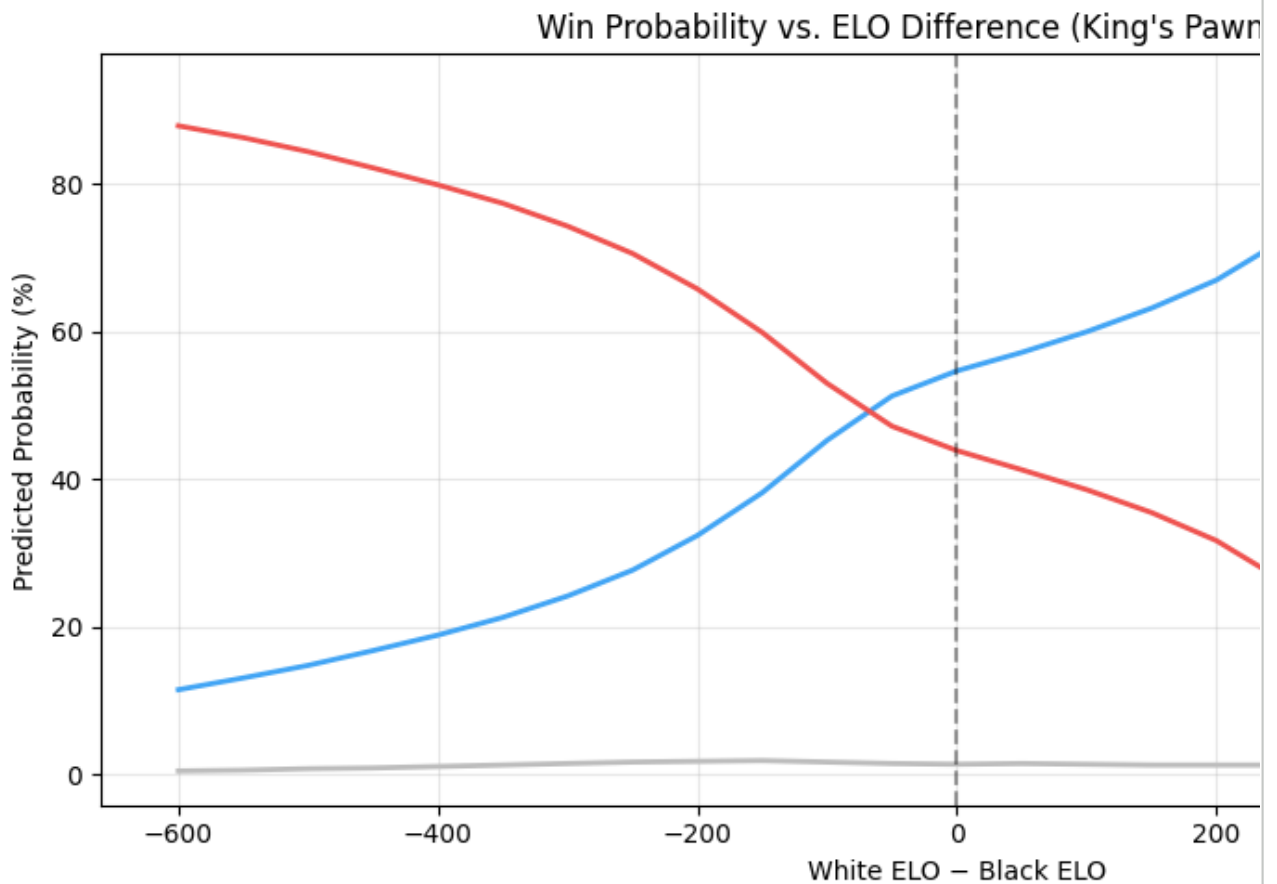
plt.plot(elo_diffs, black_win_probs, label="Black Wins", color="#FF5555")
plt.plot(elo_diffs, draw_probs, label="Draw", color="#BDBDBD", linewidth=2)
plt.axvline(0, color="black", linestyle="--", alpha=0.4, label="Equal ELO")
plt.xlabel("White ELO - Black ELO")
plt.ylabel("Predicted Probability (%)")
plt.title("Win Probability vs. ELO Difference (King's Pawn Game)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("win_probability_curve.png", dpi=150)
plt.show()
print("Win probability curve saved.")

```

```

print("\n✅ All done! Saved files: eda_plots.png, training_curves.png,

```



```

Win probability curve saved.

```

```

✅ All done! Saved files: eda_plots.png, training_curves.png, confusion_matrix.png

```

